UNIVERITSY OF CYBERJAYA

PROJECT NAME: NIXON DATATHON 2026
TEAM MEMBERS: 1. DACHAINI A/P MURUGAIYIAH
2. LOSHENNI A/P MURUGIAH
3. DRIPTIKAA A/P BUVANESWARAN
4. THIRUSELVAN A/L BASKARAN

PROJECT REPORT / TECHNICAL DOCUMENTATION

IT TROUBLESHOOTING ASSISTANT

Programming Language: Python

1. Introduction and Problem Background

Computers are important tools for students because they are used for assignments, online learning, research, and presentations. However, students may experience common computer problems, such as Wi-Fi connection issues, applications freezing, computers failing to start, and printers not working properly.

When these problems occur, some students may not know how to identify the problem or find a suitable solution. They may need to wait for technical support, which can interrupt their studies and waste time.

To address this problem, the IT Troubleshooting Assistant was developed using Python. It is a menu-driven program that provides troubleshooting guidance for common computer issues. Users can browse available problems, select a category, or search using keywords to find relevant solutions.

The system displays step-by-step instructions and safety reminders to help users troubleshoot problems more independently. It also allows users to provide feedback on whether the recommended steps solved their problems.

2. Project Objectives and Target Users

2.1 Project Objectives

The objectives of this project are:

1. To develop a Python-based system that helps users troubleshoot common computer problems.
2. To provide step-by-step guidance for hardware, software, network, performance, and peripheral issues.
3. To allow users to search for problems using keywords and browse problems by category.
4. To reduce the need for technical support when dealing with basic computer problems.
5. To provide safety reminders and collect user feedback after displaying solutions.

2.2 Target Users

The main target users are students who use computers for their studies and assignments. The system can also help lecturers, teachers, and general computer users who need basic troubleshooting guidance.

The system is designed for users with limited technical knowledge because its instructions are presented in simple language and organised into clear steps.

3. Proposed Solution and Features

The proposed solution is a Python-based IT Troubleshooting Assistant that provides a simple way to find solutions to common computer problems. The program uses a predefined collection of troubleshooting information stored in a Python dictionary named ISSUES.

The system includes the following features.

3.1 Main Menu

The main menu allows users to select one of four options:

- Browse all problems.
- Browse by category.
- Search for a problem.
- Exit the program.

The menu repeats until the user chooses to exit. Invalid selections are handled by displaying an error message.

3.2 Browse All Problems

Users can view all eight troubleshooting problems available in the system. Each problem is displayed with its number, category, and name. Users can select a problem number to view its recommended solution.

3.3 Browse by Category

The system organises troubleshooting problems into five categories:

- **Hardware:** Computer will not turn on and computer screen is blank.
- **Network:** Wi-Fi is not connecting and connected to Wi-Fi but no internet.
- **Software:** Application is freezing and application will not open.
- **Performance:** Computer is running slowly.
- **Peripheral:** Printer is not printing.

Users can select a category and view the problems related to it.

3.4 Search for a Problem

The search function allows users to enter keywords related to their computer problems. The program checks the problem names, categories, and predefined symptom keywords to find matching results.

For example, entering the keyword "printer" can display the problem "Printer is not printing". If no matching problem is found, the system informs the user and suggests trying another keyword or contacting IT support.

3.5 Step-by-Step Troubleshooting Solutions

The system displays recommended troubleshooting steps for the selected problem. Each instruction is numbered to help users follow the process in the correct order.

The solutions cover basic actions such as checking connections, restarting applications, checking device settings, and contacting technical support when necessary.

3.6 Safety Reminders

Each troubleshooting problem includes a safety warning. These reminders help users avoid unsafe actions, such as opening a power supply, handling exposed electrical components, or downloading software from unknown websites.

3.7 User Feedback

After displaying a solution, the program asks users whether the instructions solved their problem. Users can select one of three responses: Yes, Partially, or No.

The program then displays a suitable message based on the selected response. In the current implementation, the feedback is processed during the session but is not permanently stored.

3.8 Input Validation

The program checks user input to handle invalid menu selections, incorrect problem numbers, unavailable categories, empty search keywords, and invalid feedback choices. This helps prevent users from selecting options that the system cannot process.

4. System Design and Program Logic

4.1 System Design

The system uses a simple modular design in which each function performs a specific task. The main components are:

- **User Interface:** Displays the menu and receives user input.
- **Menu Management:** Processes the user's selected option.
- **Search Function:** Finds problems using keywords.
- **Category Function:** Displays available categories and their related problems.
- **Troubleshooting Data:** Stores problem names, categories, symptom keywords, solutions, and safety warnings.
- **Solution Output:** Displays troubleshooting instructions for the selected problem.
- **Feedback Function:** Collects a response and displays a suitable message.

The system uses a dictionary called ISSUES to store the troubleshooting information. Each problem has a unique number and contains its category, name, symptoms, steps, and warning.

4.2 Program Logic

The program begins by displaying a welcome message and the main menu. The user chooses an option, and the system processes the selection.

If the user chooses to browse all problems, the program displays the available problems and allows the user to select one. If the user chooses to browse by category, the program displays the categories and lists the related problems.

If the user selects the search option, the program requests a keyword and searches the stored troubleshooting information. When matching problems are found, the results are displayed for selection. If no match is found, an appropriate message is shown.

After a valid problem is selected, the system displays the troubleshooting steps and safety reminder. The user then provides feedback about the solution.

Finally, the program returns to the main menu so the user can continue troubleshooting or select the exit option to terminate the application.

4.3 Main Functions

The program uses the following Python functions:

Function	Purpose
show categories ()	Displays the available troubleshooting categories.
show issues (category=None)	Displays all problems or filters them by category.
search issues (keyword)	Searches for matching problems using keywords.
give solution (number)	Displays the solution, safety reminder, and feedback options.
run assistant ()	Runs the main menu and controls the overall program flow.

These functions make the program easier to understand, organise, test, and maintain.

5.0 Implementation Details

5.1 Programming Language

The IT Troubleshooting Assistant is implemented using Python. Python was selected because its syntax is readable and it supports useful data structures and functions for building a menu-driven application.

5.2 Data Storage

The system stores its troubleshooting information in a Python dictionary named ISSUES. The dictionary contains eight predefined problems. Each problem is associated with a unique key and stores several pieces of information, including its category, name, symptoms, troubleshooting steps, and safety warning.

This approach allows the program to retrieve the relevant information whenever a user selects a problem.

The current version uses data stored directly in the source code. It does not require an external database.

5.3 Searching Mechanism

The search issues () function converts the entered keyword to lowercase and removes unnecessary spaces at the beginning and end. It then compares the keyword with the problem name, category, and symptom keywords.

When a match is found, the problem is added to the search results. The matching results are then displayed to the user.

5.4 Category Filtering

The show categories () function identifies the categories stored in the dictionary and displays them in alphabetical order.

The show issues () function can display all problems or only those belonging to a selected category. The category comparison ignores differences in uppercase and lowercase letters.

5.5 Displaying Solutions and Feedback

The give solution () function retrieves the selected problem and displays its name, category, numbered troubleshooting steps, and safety reminder.

The function then asks the user to provide feedback. A while loop ensures that the user enters a valid response of 1, 2, or 3 before the program displays the corresponding feedback message.

5.6 Program Control and Validation

The run assistant () function controls the main program using a while loop and conditional statements. The loop keeps the application running until the user selects the exit option.

The program uses if, Elif, and else statements to process menu selections. It also validates problem numbers, categories, search keywords, and feedback choices to provide appropriate responses to invalid input.

Overall, the implementation combines Python dictionaries, functions, loops, conditional statements, string operations, and input validation to provide a basic troubleshooting assistant.

6. Testing Results

Testing is necessary to check whether the system performs its intended functions correctly. The following test cases are based on the implemented program and should be executed to confirm the actual results.

No.	Test Case	Expected Result
1	Start the program	The welcome message and main menu are displayed.
2	Select Browse all problems	All eight troubleshooting problems are displayed.
3	Browse the Hardware category	The two hardware problems are displayed.
4	Search using the keyword "printer"	The printer troubleshooting problem is displayed as a match.
5	Search using an unknown keyword	A no-matching-problem message is displayed.
6	Enter an invalid menu option	An error message asks the user to enter a valid option.
7	Select a valid problem	The problem details, troubleshooting steps, and safety reminder are displayed.
8	Enter an invalid feedback choice	The program asks the user to enter 1, 2, or 3 again.
9	Select the wrong problem number for a category	The program displays an appropriate validation message.
10	Select the Exit option	A goodbye message is displayed and the program terminates.

Testing status: The expected results are derived from the code's logic. The test cases must be run against the program, and the actual outcomes should be recorded before this table is presented as completed testing.

7. Limitations and Future Enhancements

7.1 Limitations

Although the system provides useful troubleshooting guidance, it has several limitations.

First, it contains only eight predefined problems. Users may not find a solution if their computer issue is not included in the dictionary.

Second, the search function uses keyword matching rather than advanced language understanding. Therefore, it may not recognise every way a user describes a problem.

Third, the troubleshooting information is stored directly in the Python source code. Adding or updating problems requires changes to the code.

Fourth, the system provides general instructions and cannot automatically diagnose the actual condition of a computer or confirm that a solution has fixed the issue.

Finally, the feedback feature displays a response but does not permanently save feedback for later analysis. The current program also uses a text-based interface rather than a graphical interface.

7.2 Future Enhancements

Several improvements can be introduced in future versions.

1. **Expand the troubleshooting database:** Add more hardware, software, network, and peripheral problems.
2. **Improve keyword searching:** Introduce more flexible search methods to recognise different descriptions of the same problem.
3. **Develop a graphical user interface:** Create a more attractive and user-friendly interface with buttons, icons, and category menus.
4. **Add permanent feedback storage:** Save feedback in a file or database so that it can be reviewed later.
5. **Use an external database:** Store troubleshooting information separately from the program to make updates easier.
6. **Add multilingual support:** Allow users to access troubleshooting instructions in different languages.
7. **Introduce advanced recommendations:** Suggest possible solutions based on the symptoms described by the user.

These enhancements could improve the system's usability, flexibility, and usefulness for students.

8. Conclusion

In conclusion, the IT Troubleshooting Assistant is a Python-based application designed to help students and other computer users find solutions to common computer problems. The system provides eight predefined troubleshooting guides organised into five categories: Hardware, Network, Software, Performance, and Peripheral.

Its main functions include browsing problems, searching with keywords, displaying step-by-step instructions, providing safety reminders, and collecting user feedback. The use of Python functions, dictionaries, loops, and conditional statements helps organise the program and manage user interactions.

Although the current version has limitations, such as a small troubleshooting collection, basic keyword matching, and no permanent feedback storage, it provides a useful foundation for a troubleshooting support system. Future improvements could make it more flexible and user-friendly.

Overall, the project demonstrates how programming can be used to develop a practical tool that helps users handle basic computer problems and become more independent when troubleshooting technical issues.